

From Dense Retrieval to Retrieval-Augmented Generation

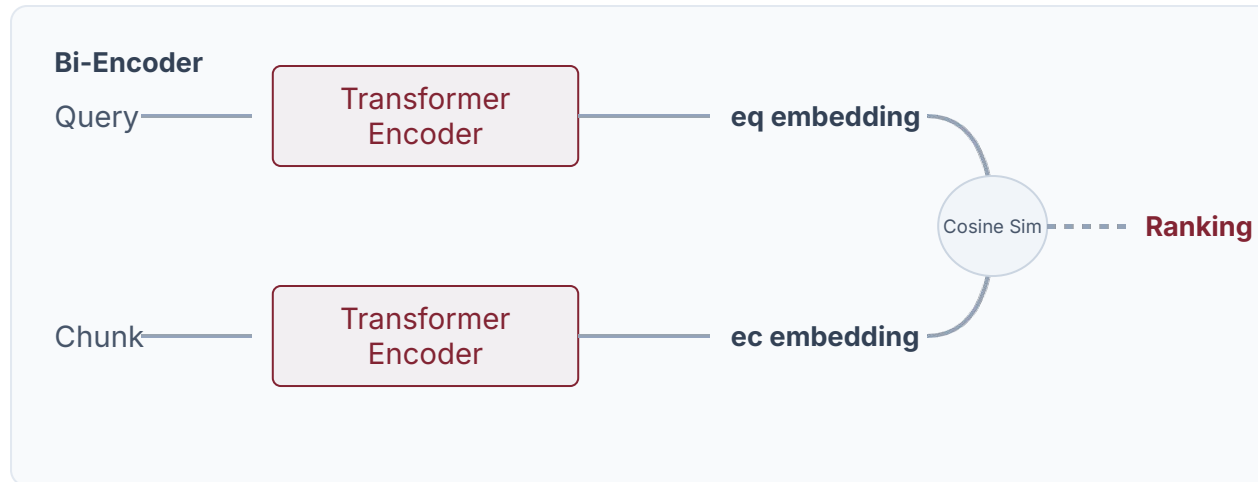
MNLP Homework 1 & Homework 2

Presenter: Antonio Serra

How retrieval quality affects factual answer generation in small language models.

Theoretical Background: Dense Retrieval

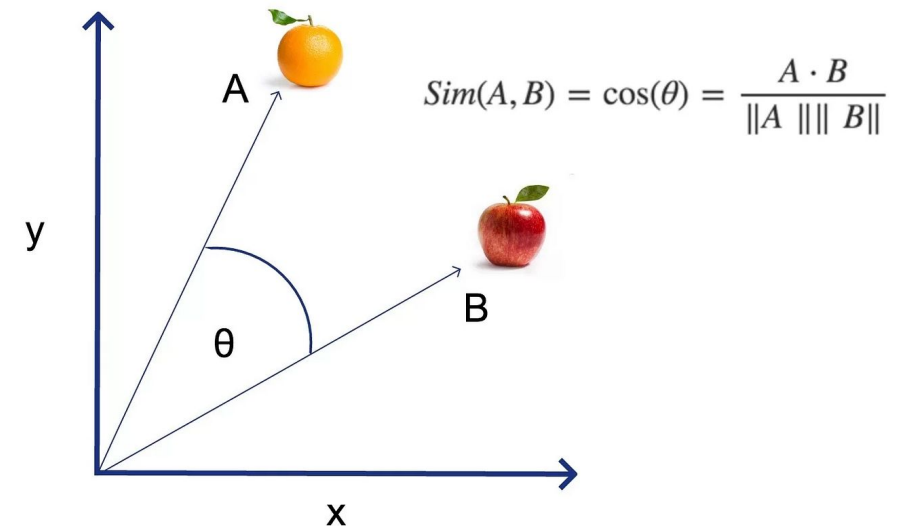
Dense retrieval maps natural-language queries and candidate passages into the same continuous vector space, so that semantic relevance can be estimated geometrically.



Why mean pooling?

- Encoders produce contextual representations for **every token**.
- Mean pooling **aggregates** these vectors into a single sentence-level embedding.
- The **attention mask** is used to ignore padding tokens, ensuring the final vector represents only meaningful content.

Cosine Similarity



"Higher cosine similarity means that the query and the chunk are closer in the embedding space and therefore more likely to be semantically related."

Key idea: retrieval is transformed into a semantic ranking problem over dense vector representations.

HW1 Retrieval Pipeline

Implemented Pipeline

- 1 **Tokenization** of text strings.
- 2 Forward pass through **Transformer encoder**.
- 3 **Mean pooling** with attention mask applied.
- 4 **Cosine similarity** computation.
- 5 **Ranking** over all candidate chunks.
- 6 **Evaluation** across validation data.

$$\text{Hit@K} = \frac{\sum_{i=1}^{|Q|} \text{hit}_i}{|Q|}$$

Evaluation Metrics

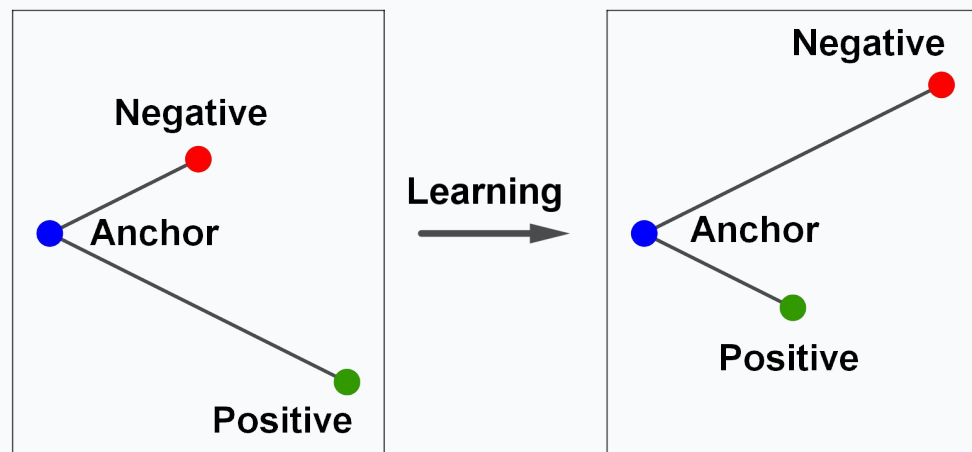
Hit@k: Measures whether the correct chunk is present within the top-k retrieved results (binary 0 or 1). It primarily evaluates basic retrieval success.

MRR (Mean Reciprocal Rank): Averages the inverse of the rank of the first correct chunk. It heavily penalizes systems that place the correct chunk lower down the list, rewarding precise ranking.

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Supervised Fine-Tuning: Triplet Strategy

Triplet Schematic



Triplet components

Anchor: the input query.

Positive: the gold chunk containing the correct evidence.

Negative: an incorrect candidate chunk used as contrastive example.

Random negative

Fast & Stable signal

Low computational cost

Lower risk of false negatives

Hard negative

Slower & Stronger contrast

Extra encoding/similarity

High risk of ambiguity

In our final setting, random negatives were preferred because they reduced training time and avoided noisy hard negatives that were too semantically close to the gold evidence.

HW1 Experimental Setup and Retrieval Results

Setup

Models & Parameters

- DistilBERT: 66M
- MiniLM: 22M
- TinyBERT: 14M

Configuration (T4 GPU)

- Batch: 32 | Optimizer: AdamW | Epochs: 1
- LR: 2e-5 (DistilBERT/MiniLM), 1e-5 (TinyBERT)

Training Loss

- **DistilBERT:** 0.1196 | **MiniLM:** 0.0979 | **TinyBERT:** 0.1900

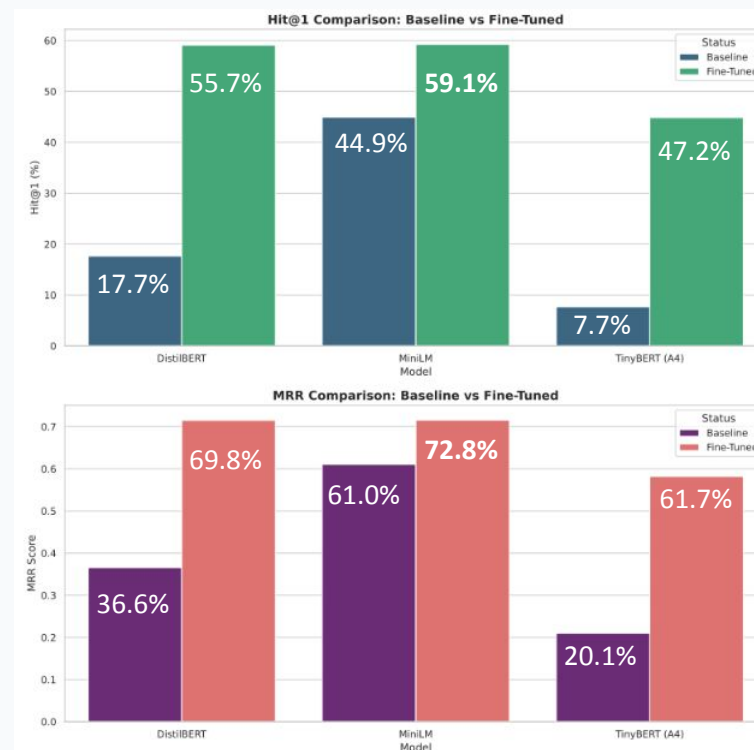
Training Time (~Random / ~Hard Neg)

- DistilBERT: 6m / 40m | MiniLM: 6m / 15m | TinyBERT: 6m / 12m

Note:

MiniLM was tested in two configurations: AutoModel + manual mean pooling and the native SentenceTransformers interface. The SentenceTransformers configuration achieved Hit@1 = 0.4480 and MRR = 0.6093, almost identical to the AutoModel baseline.

Hit@1 and MRR: Baseline vs Fine-Tuned



What do the metrics capture? **Hit@k:** Measures if the gold chunk is in the top-k passages (binary success). **MRR:** Measures rank position of the gold chunk; rewards higher ranks (1/rank).

Takeaway: Fine-tuning strongly improves retrieval quality across all models; MiniLM remains the strongest overall, while TinyBERT shows that even compact models benefit substantially from task-specific supervision.

Why RAG? Limits of Parametric Knowledge

- LLMs store knowledge implicitly in internal parameters.
- This knowledge is **static, opaque, lossy**, and difficult to update.
- Small LLMs have limited factual coverage due to drastically reduced parameter counts.
- **Retrieval-Augmented Generation (RAG)** bypasses this by injecting external, relevant knowledge at inference time.



Parametric Knowledge

Knowledge stored implicitly inside model weights during massive pre-training phases.



Non-Parametric Knowledge

Explicit textual evidence accurately retrieved from an external Knowledge Base on-the-fly.

HW2 RAG Architecture: Evidence-Grounded Generation



HW2 extends HW1: retrieval is no longer the final goal, but the first stage of factual answer generation.

Generation Settings: Baseline, RAG, Oracle and Wikidata-RAG

1. Baseline

The model receives only the question. This tests the LLM's parametric knowledge.

Prompt Skeleton

Question: {question}
Answer shortly:

No retrieved evidence is provided.

2. RAG

The model receives the top-3 chunks retrieved by the fine-tuned MiniLM retriever.

Prompt Skeleton

Given the following information:
[1] {retrieved_chunk_1}
[2] {retrieved_chunk_2}
[3] {retrieved_chunk_3}
Question: {question}
Answer shortly:

Generation is grounded on retrieved non-parametric evidence.

3. Oracle

Oracle estimates the effect of always providing the gold evidence in the first position.

Prompt Skeleton

Given the following information:
[1] {gold_chunk}
[2] {retrieved_chunk_1}
[3] {retrieved_chunk_2}
Question: {question}
Answer shortly:

Useful to separate retrieval errors from generation errors.

4. Wikidata-RAG

Wikidata labels and descriptions are injected as additional entity-level context.

Prompt Skeleton

Additional Knowledge (Wikidata):
{wikidata_description}
Given the information above and the context below:
[1] {retrieved_chunk_1}
[2] {retrieved_chunk_2}
[3] {retrieved_chunk_3}
Question: {question}
Answer shortly:

Wikidata acts as a semantic anchor for disambiguation.

Why compare these settings?

Baseline measures internal knowledge. RAG measures retrieved evidence effect. Oracle estimates perfect retrieval impact. Wikidata-RAG tests structured entity metadata grounding.

The amount and quality of external context directly shape the factuality of the generated answer.

HW2 Evaluation Metrics: What Do We Measure?

Automatic metrics

Exact Match / EM

Definition: EM checks whether the generated answer exactly matches the gold short answer after normalization.

Gold: Albert Einstein

Gen: Albert Einstein → EM=1

Gen: Einstein → EM=0

Captures: Very strict correctness.

Limitation: Penalizes valid paraphrases.

Substring EM / subEM

Definition: Checks if the gold answer appears as a substring inside the generated answer.

Gold: Albert Einstein

Gen: The answer is Albert Einstein. → subEM=1

Gen: Einstein → subEM=0

Captures: Containment of correct short answer.

Limitation: Based on surface string matching.

METEOR

Definition: Measures lexical similarity considering word overlap and softer matching.

Gold: United States

Gen: the United States of America

→ Partial/high score

Captures: Approximate textual similarity.

Limitation: Can reward overlap without factuality.

Human and LLM-based evaluation

LLM-as-a-Judge

Def: Binary score (1=valid form, 0=else).

Captures: Try to capture reliable factual correctness

Gold: Paris | **Gen:** Capital is Paris. | **Score:** 1

Manual validation

Def: Human annotators inspect using the same binary criterion.

Captures: Reliable factual correctness.

Gold: Rome | **Gen:** Italy | **Score:** 0 (Related, not correct)

Cohen's Kappa

Def: Measures agreement beyond chance.

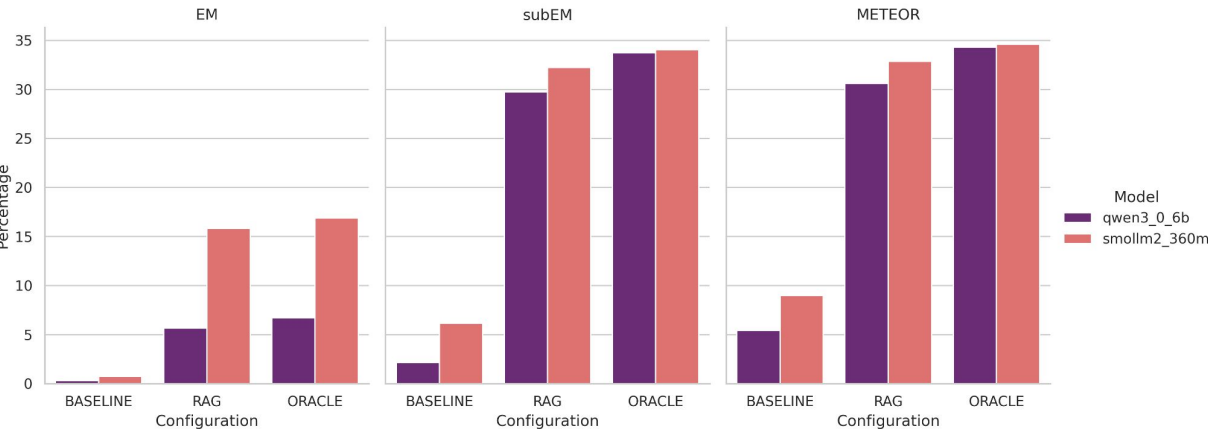
Captures: How consistently two evaluators assign the same labels after accounting for chance agreement.

Automatic metrics are useful for large-scale comparison, but manual validation is essential to assess factual correctness in short-answer RAG.

HW2 Results: RAG Effect, Wikidata Enrichment and Evaluation Reliability

Automatic evaluation on test split

Model	Setting	EM (%)	subEM (%)	METEOR (%)
Qwen-0.6B	Baseline	0.30	2.15	5.41
Qwen-0.6B	RAG	5.65	29.75	30.63
Qwen-0.6B	Oracle (upper)	6.70	33.75	34.33
SmolLM2-360M	Baseline	0.75	6.15	8.96
SmolLM2-360M	RAG	15.85	32.25	32.88
SmolLM2-360M	Oracle (upper)	16.90	34.05	34.62



Wikidata enrichment on SmolLM2 RAG

Configuration	EM (%)	subEM (%)	METEOR (%)
Standard RAG	15.85	32.25	32.88
Wikidata	15.75	32.70	33.06

Wikidata slightly improves subEM and METEOR, but slightly decreases strict EM. This suggests a small increase in lexical overlap or answer elaboration, while the additional entity descriptions may also introduce prompt noise.

Judge positive rate: **96.50%** Human positive rate: **56.5%**
(Based on 200 random samples)

LLM-as-a-Judge reliability

Comparison	Cohen's Kappa	Interpretation
Human 1 vs Human 2	0.9898	Almost perfect
Human 1 vs LLM Judge	0.0672	Very low
Human 2 vs LLM Judge	0.0688	Very low

Oracle improvements show that retrieval quality remains one limiting factor, while the qualitative analysis also reveals substantial generation failures even when the gold evidence is available.

A1 and A3 – Context Processing and Knowledge Enrichment

All results are obtained with SmoLM2-360M-Instruct using the fine-tuned MiniLM retriever with top-3 chunks.

Methods Overview

Standard RAG

The generator receives the top-3 retrieved chunks together with the question.

Compressed RAG

Each retrieved chunk is split into sentences. MiniLM embeddings and cosine similarity are used to keep only the most query-relevant sentences while preserving their original order.

Corrective RAG

The retrieved context is unchanged, but the generation prompt is stricter. The model answers only when the evidence is sufficient; otherwise, it can return "Unknown".

Wikidata-Enriched RAG

Wikidata labels and descriptions associated with the entity identifier are added to the retrieved context before generation.

Best Overall Variant

Compressed RAG achieved the best EM and METEOR by reducing irrelevant context before generation.

Conservative Prompting

Corrective RAG obtained the best subEM and produced 13 "Unknown" answers (0.65% of the test set), but did not improve overall performance.

Limited Wikidata Gain

Wikidata slightly improved subEM and METEOR but slightly reduced EM, suggesting that external entity descriptions can add both useful information and prompt noise.

Results

Configuration	EM (%)	subEM (%)	METEOR (%)
Standard RAG	15.85	32.25	32.88
Compressed RAG	17.80	32.60	33.33
Corrective RAG	15.60	32.80	32.36
Wikidata-Enriched RAG	15.75	32.70	33.06
Compressed RAG provides the strongest overall performance.			

For SmoLM2-360M-Instruct, sentence-level context compression was more effective than stricter prompting or Wikidata enrichment.

Alternative LLM Judges

Judge	Positive rate	Cohen's κ vs Human 1	Cohen's κ vs Human 2
SmolLM2 original judge	96.5%	0.0672	0.0688
Qwen2.5-3B-Instruct	38.5%	0.5728	0.5640
Qwen2.5-1.5B-Instruct	53.5%	0.8183	0.8081

Permissiveness

The original SmolLM2 judge was excessively permissive, severely overestimating generation quality compared to human annotations.

Moderate Alignment

Qwen2.5-3B-Instruct showed moderate agreement with human annotators, substantially improving over the baseline judge.

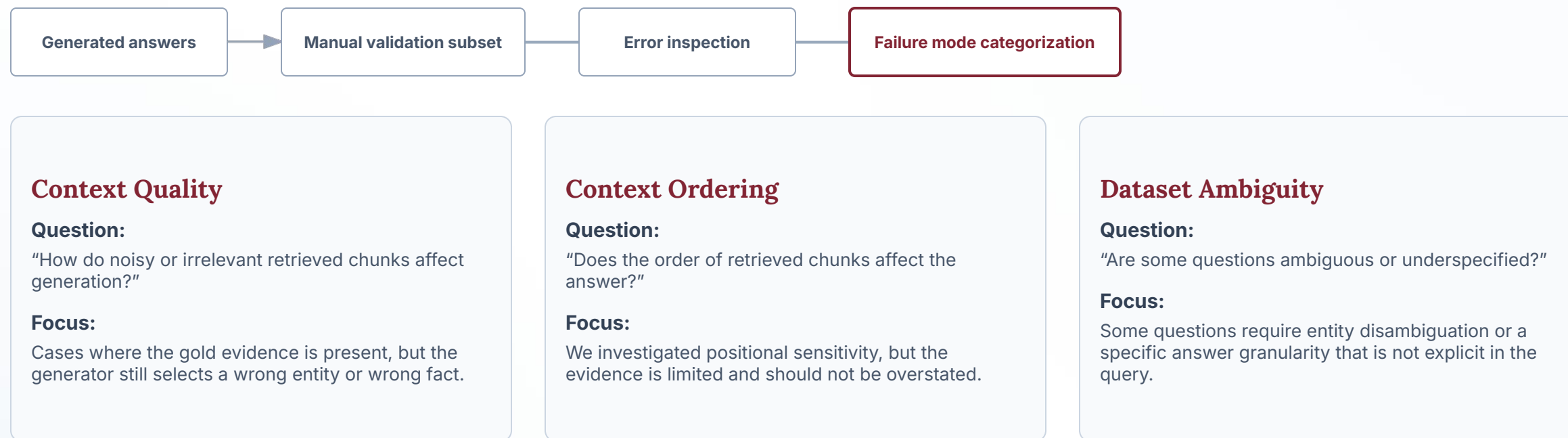
Highest Agreement

Qwen2.5-1.5B-Instruct achieved the highest agreement with both humans, demonstrating strong reliability despite its small scale.

Judge selection substantially changes the evaluation outcome. Qwen2.5-1.5B-Instruct was the most reliable automatic judge in our experiments.

Qualitative Analysis: Post-hoc Error Inspection

We performed a manual post-hoc qualitative analysis on the same 200-sample subset used for LLM-as-a-Judge and human validation.



Qualitative analysis helps distinguish retrieval errors, generation errors, context noise, and answer-type or granularity mismatches

Context Quality: When Retrieved Evidence Becomes Distracting

Manual subset
200
examples

Failures with gold evidence present
62

This indicates that some errors are not caused by missing retrieval alone. The **model may see the correct evidence but still select a more salient, distracting, or same-type entity** from another chunk.

Representative failure patterns

Question	Gold answer	Model answer	Failure pattern
"who played the mom on lost in space" (query_id = 6cMFY1k9RzLR)	June Lockhart	Anne Lockhart	<i>Same surname, semantic type, distracting family entity.</i>
"who played bass on oh what a night" (query_id = 9vIDB8PI8kN1)	Don Ciccone	Bob Gaudio and Judy Parker	<i>Role confusion: bass player vs songwriters.</i>
"who wrote the lyrics to the first time ever i saw your face" (query_id = jWMqhHPRF7nQ)	Ewan MacColl	Roberta Flack	<i>Salient performer selected instead of songwriter.</i>
"what hotel was the las vegas shooting in" (query_id = ty36AvWsFskb)	Mandalay Bay Hotel	The Las Vegas Strip	<i>Location granularity error: broader location instead of specific hotel.</i>

Main interpretation

Noisy chunks are especially harmful when they contain entities of the **same type** as the correct answer: person vs person, location vs location, date vs date.

The generator may copy the **most salient entity** rather than the correct one found in the gold evidence.

The bottleneck is not only retrieval: generation can fail even with relevant evidence available, due to context noise and entity confusion.

Qualitative Analysis: Context Ordering and Dataset Ambiguity

Context ordering: weak evidence of positional sensitivity

Evidence position	Accuracy	Samples
Position 0	63.20%	125
Position 1	62.50%	24
Position 2	53.33%	15
No gold chunk found	30.56%	36

*Position 0 and Position 1 are almost identical.
Position 2 is lower, but the sample size is small.*

We therefore do not claim strong positional bias. The safer conclusion is that **retrieval quality matters more than ordering**, while ordering may affect a few isolated cases.

Dataset ambiguity: when the expected answer is under-specified

Some questions do not fully specify the expected entity type or level of granularity. The model may return a semantically related answer that is plausible, but does not match the annotated short answer.

Ambiguity A: entity type mismatch

Given the following information:

[1] {retrieved_chunk_1...}
[2] {retrieved_chunk_2...}
[3] {retrieved_chunk_3...}

Question: who played the mom on lost in space

Gold: june lockhart

Model: Anne Lockhart

(query_id = 6cMFY1k9RzLR)

The model selects a related person with the same surname instead of the correct actress. This is a generation error caused by distracting contextual entities, not an ambiguity in the expected entity type.

Ambiguity B: location granularity

Given the following information:

[1] {retrieved_chunk_1...}
[2] {retrieved_chunk_2...}
[3] {retrieved_chunk_3...}

Question: where is the biggest guinness brewery in the world

Gold: Ireland

Model: St. James' Gate Brewery

(query_id = Ms8VBqNxxnXY)

The question asks 'where', but the expected answer may be a country while the model outputs a more specific locality. The answer is related, but has the wrong granularity for strict evaluation.

Ambiguity C: actor vs character

Given the following information:

[1] {retrieved_chunk_1...}
[2] {retrieved_chunk_2...}
[3] {retrieved_chunk_3...}

Question: who plays the blackfish in game of thrones

Gold: clive russell

Model: Brynden "The Blackfish" Tully

(query_id = MLd2XozzEbHz)

The question is formulated with 'who', but it is not fully explicit whether the expected answer is the actor/person or the fictional character. The model returns the character, while the gold answer expects the actor.

Dataset ambiguity interacts with strict short-answer evaluation: a generated answer can be semantically plausible but still be counted as wrong because the expected entity type or granularity is not explicit in the question.

Key Lesson: Beyond Algorithmic Performance

RAG strongly improves small LMs, but reliable evaluation and effective use of retrieved evidence remain major challenges.

Thank You!